Day 139 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 7 reference

Watch: The diffusion model (DDPM) – the basics | Generative AI (TileStats) | What are Diffusion Models? (Ari Seff)

`day139_gan_build_and_forward_diffusion.py`, segment by segment, in the order `main()` uses things. The 43 functions and classes of day133_exercise.py to day138_exercise.py are copied in verbatim and are explained in those days' guides; this walkthrough covers what is new today: the noise schedule, the forward process, the checks, the week's training loop and its scoring, and the experiments. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: t = step (0–1,000) · β_t = noise added at step t · A_t = abar[t] = $\prod_{s \le t}(1 - \beta_s)$ · $\varepsilon \sim N(0, I)$

Segment 0 — Setup

One file for the whole week

CODE

```
d134 = d136 = sys.modules[__name__]
```

- `sys.modules[__name__]` — the module object of this very file. The copied parts call each other as d134.gan_step or d136.Conv; pointing both names at this file makes those calls find the copies here.
- `a = b = value` — chained assignment: both names refer to the same object.

Segment 1 — The forward diffusion process

schedule: how much noise each step adds

MATH

linear: β_t *from 0.0001 to 0.02;* $\quad$ *cosine:* $A_t = \cos^2((t/T + 0.008)/1.008 \cdot \pi/2) \,/\, \cos^2(0.008/1.008 \cdot \pi/2)$

CODE

```
def schedule(kind="linear", T=1000):
    if kind == "linear":
        betas = np.linspace(1e-4, 0.02, T)
    else:
        f = lambda t: np.cos((t / T + 0.008) / 1.008 * np.pi / 2) ** 2
        abar = f(np.arange(T + 1)) / f(0); betas = np.clip(1 - abar[1:] / abar[:-1], 0, 0.999)
    abar = np.concatenate([[1.0], np.cumprod(1 - betas)])
    return betas, abar
```

- `np.linspace(1e-4, 0.02, T)` — T evenly spaced values from 0.0001 to 0.02: Ho et al.'s linear schedule.
- `f = lambda t: np.cos(...) ** 2` — the cosine schedule (Nichol and Dhariwal, 2021) defines A_t directly, as a squared cosine of t.
- `1 - abar[1:] / abar[:-1]` — each β recovered from neighbouring A values: [1:] drops the first element, [:-1] the last, so they line up as (t, t – 1).
- `np.cumprod(1 - betas)` — the running product: A_1, A_2, ...; [1.0] is put in front so abar[0] = 1 (no noise yet).

q_step and q_sample: one step, or one jump

MATH

$$x_t = \sqrt{(1-\beta_t)}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon \;\Rightarrow\; x_t = \sqrt{A_t}\, x_0 + \sqrt{(1-A_t)}\, \varepsilon$$

CODE

```
def q_step(x, beta, rng):
    return np.sqrt(1 - beta) * x + np.sqrt(beta) * rng.standard_normal(x.shape)
def q_sample(x0, t, abar, rng):
    return np.sqrt(abar[t]) * x0 + np.sqrt(1 - abar[t]) * rng.standard_normal(x0.shape)
```

- `np.sqrt(1 - beta) * x + np.sqrt(beta) * noise` — shrink the signal so its variance drops by β, then add noise of variance β: a variance of 1 stays 1.
- `rng.standard_normal(x.shape)` — fresh N(0, 1) noise, one number per value of x.
- `np.sqrt(abar[t]) * x0 + ...` — the same result as t small steps, in one line: the steps' noises add up to one Gaussian.

Segment 2 — The checks

demo_checks: the parts are really verbatim

CODE

```
    me = open(os.path.abspath(__file__), encoding="utf-8").read(); found, total = 0, 0
    for d in range(133, 139):
        src = open(os.path.join(HERE, f"day{d}_exercise.py"), encoding="utf-8").read(); lines =
            src.split("\n")
        for node in ast.parse(src).body:
            if isinstance(node, (ast.FunctionDef, ast.ClassDef)):
                total += 1; found += "\n".join(lines[node.lineno - 1:node.end_lineno]) in me
```

- `open(os.path.abspath(__file__))` — the script reads its own source code.
- `ast.parse(src).body` — Python's parser turns each exercise file into a tree; .body is its top-level statements.
- `node.lineno, node.end_lineno` — the first and last line of each function or class.
- `found += (text in me)` — True counts as 1: the number of parts whose exact text appears in this file.

demo_checks: 1,000 small steps against one jump

MATH

$$E[x_t\, x_0] = \sqrt{A_t}\, E[x_0^2] = \sqrt{A_t} \text{ when } x_0 = \pm 1$$

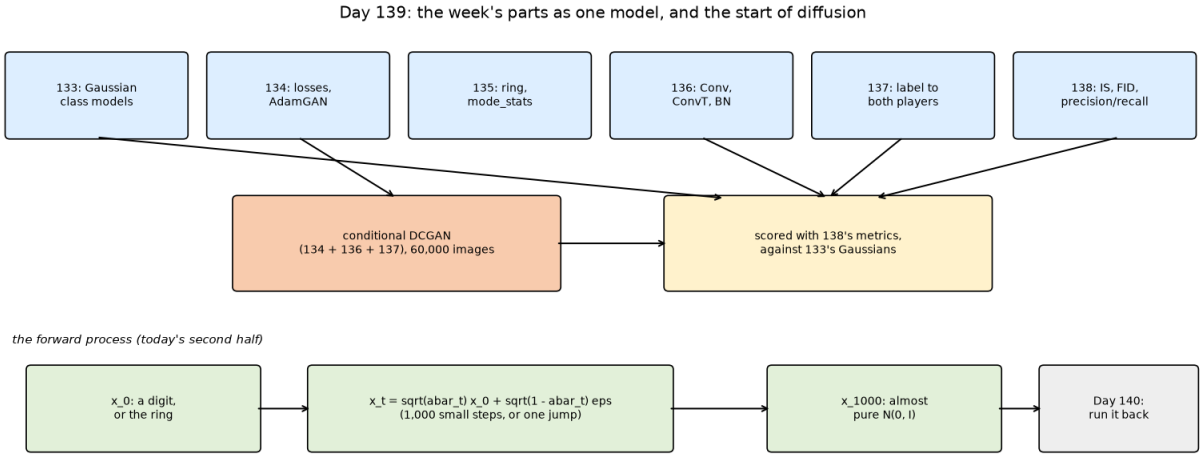
CODE

```
rng = np.random.RandomState(0); x0 = np.where(rng.rand(200000) < 0.5, -1.0, 1.0)
x = x0.copy(); print(f"{'t':>6} {'1,000-step route: mean, variance, E[x x_0]':>44} {'shortcut:
    mean, variance, E[x x_0]':>37} {'formula E[x x_0] = sqrt(abar)':>30}")
for t in range(1, 1001):
    x = q_step(x, betas[t - 1], rng)
    if t in (10, 100, 300, 1000):
        j = q_sample(x0, t, abar, rng)
        print(f"{t:6d} {x.mean():14.4f} {x.var():9.4f} {(x * x0).mean():12.4f}
            {j.mean():14.4f} {j.var():9.4f} {(j * x0).mean():10.4f} {np.sqrt(abar[t]):18.4f}")
```

- `np.where(rng.rand(n) < 0.5, -1.0, 1.0)` — a coin flip per point: data of mean 0 and variance exactly 1.

- `x = q_step(x, betas[t - 1], rng)` — betas[t – 1] is step t's β (the array starts at index 0).
- `(x * x0).mean()` — how much of the original signal is left: it should equal $\sqrt{A_t}$.

SYSTEM DIAGRAM



The week's parts, and the forward process.

Segment 3 — The week's model

train_week: the conditional DCGAN on all 60,000 images

CODE

```
def train_week(X, y, sc, epochs, seed, at, k=64, batch=128):
    G, D = cgan_players(seed, k)
    oG, oD = AdamGAN(G.net, 2e-4, 0.5), AdamGAN(D.net, 2e-4, 0.5)
    rng = np.random.RandomState(seed); Zfix = np.random.RandomState(9).standard_normal((2000,
        k)).astype(np.float32)
    yfix = np.tile(np.arange(10), 200); log, t0 = [], time.perf_counter()
    for ep in range(1, epochs + 1):
        o = rng.permutation(len(X))
        for s in range(0, len(X) - batch + 1, batch):
            b = o[s:s + batch]; cgan_step(G, D, oG, oD, X[b], y[b], rng, k)
        if ep in at:
            F = net_generate(G, g_input(Zfix, yfix)); log.append((ep, sc(F), asked(sc, F, yfix),
                time.perf_counter() - t0))
    return G, log, F, yfix
```

- `cgan_players(seed, k)` — Day 137's part, which builds Day 136's convolutional players with room for the label.
- `cgan_step(G, D, oG, oD, X[b], y[b], rng, k)` — Day 137's round of the conditional game, unchanged.
- `sc(F), asked(sc, F, yfix)` — Day 138's five scores, and whether each image shows the digit asked for.

asked and per_digit_fid

CODE

```
def asked(sc, F, labels):
    said = np.concatenate([sc.head.forward(sc.feats(F[i:i + 1000])).argmax(1) for i in range(0,
        len(F), 1000)])
    return float(np.mean(said == labels)), np.array([np.mean(said[labels == d] == d) for d in
        range(10)])
def per_digit_fid(sc, F, labels, X, y):
    Fr = sc.Fref; yr = y[:len(Fr)]; out = []
    for d in range(10):
        mu, S = gauss(Fr[yr == d]); out.append(frechet(*gauss(sc.feats(F[labels == d])), mu, S))
    return np.array(out)
```

- `sc.head.forward(sc.feats(...)).argmax(1)` — the feature network's own digit verdict, read through the Scorer's body and head.
- `Fr[yr == d]` — the reference features of real images of digit d (training images 0–9,999).
- `frechet(*gauss(...), mu, S)` — * unpacks gauss's (mean, covariance) pair into frechet's first two arguments.

demo_week: a random 2,000 of the test set

CODE

```
pt = 60000 + np.random.RandomState(0).permutation(10000)[:2000]
real = sc(X[pt]); ra = asked(sc, X[pt], y[pt]); rpd = per_digit_fid(sc, X[pt], y[pt], X, y)
```

- `permutation(10000)[:2000]` — 2,000 of the test positions in random order: Day 138 found that MNIST's stored order is not random.
- `X[pt], y[pt]` — fancy indexing with an array of positions picks those images and their labels.

demo_gaussians: Day 133's model, sampled

CODE

```
for kind, reg in (("shared", 1e-3), ("full", 1e-3), ("full", 1e-2)):
    t0 = time.perf_counter(); m = fit_gaussians(X[:60000].astype(np.float64), y[:60000], 10, kind,
        reg); rng = np.random.RandomState(0)
    F = np.empty((2000, 784), np.float32)
    for d in range(10):
        F[labels == d] = sample_class(m, d, 200, rng)
    F = np.clip(F, 0, 1); s = sc(F); a = asked(sc, F, labels); out[(kind, reg)] = (s, a, F)
```

- `fit_gaussians(..., kind, reg)` — Day 133's part: one mean per digit, and a shared or a per-digit (full) covariance, with reg added to the diagonal.
- `F[labels == d] = sample_class(m, d, 200, rng)` — a boolean mask on the left of = fills only the rows asked to be digit d.
- `np.clip(F, 0, 1)` — a Gaussian can produce pixels below 0 or above 1; they are cut back to the valid range.

Segment 4 — The forward process, measured

demo_ring: the ring, noised

CODE

```
for t in (0, 1, 5, 10, 25, 50, 100, 200, 400, 700, 1000):
    xt = q_sample(x0, t, abar, rng); m, g, _ = mode_stats(xt); fd = frechet(*gauss(xt),
        np.zeros(2), np.eye(2))
    rows.append((t, m, g, fd, xt[:2000])); print(f"{t:6d} {np.sqrt(abar[t]):11.4f} {np.sqrt(1 -
        abar[t]):9.4f} {m:6d} {g:11.3f} {fd:22.4f}")
```

- `m, g, _ = mode_stats(xt)` — Day 135's part returns three things; _ discards the per-mode counts.
- `frechet(*gauss(xt), np.zeros(2), np.eye(2))` — Day 138's Frechet distance between x_t's fitted Gaussian and N(0, I), the process's end point.

demo_mnist_noise: is it still a digit?

CODE

```
for t in (0, 50, 100, 200, 300, 400, 500, 600, 700, 800, 1000):
    xt = q_sample(x0, t, abar, rng); img = ((xt + 1) / 2).astype(np.float32)
    ca = asked(sc, img, yt)[0]
    W, b = d116.fit_softmax(q_sample(xtr, t, abar, rng), ytr, 10, lam=1e-3, steps=200, lr=0.05)
    na = float(np.mean(d116.softmax(xt @ W + b).argmax(1) == yt)); s = sc(img)
    rows.append((t, np.sqrt(abar[t]), ca, na, s[1], img[:1].copy())); print(f"{t:8d}
        {np.sqrt(abar[t]):11.4f} {ca:14.3f} {na:14.3f} {s[1]:9.2f}")
```

- `((xt + 1) / 2).astype(np.float32)` — back from [–1, 1] to the [0, 1] pixels the feature network was trained on.
- `d116.fit_softmax(..., lr=0.05)` — Day 116's softmax regression, trained on digits noised to this t; its default learning rate, 0.5, was unstable here (the run prints the tuning).
- `xt @ W + b` — the trained regression's class scores for the noised test digits.

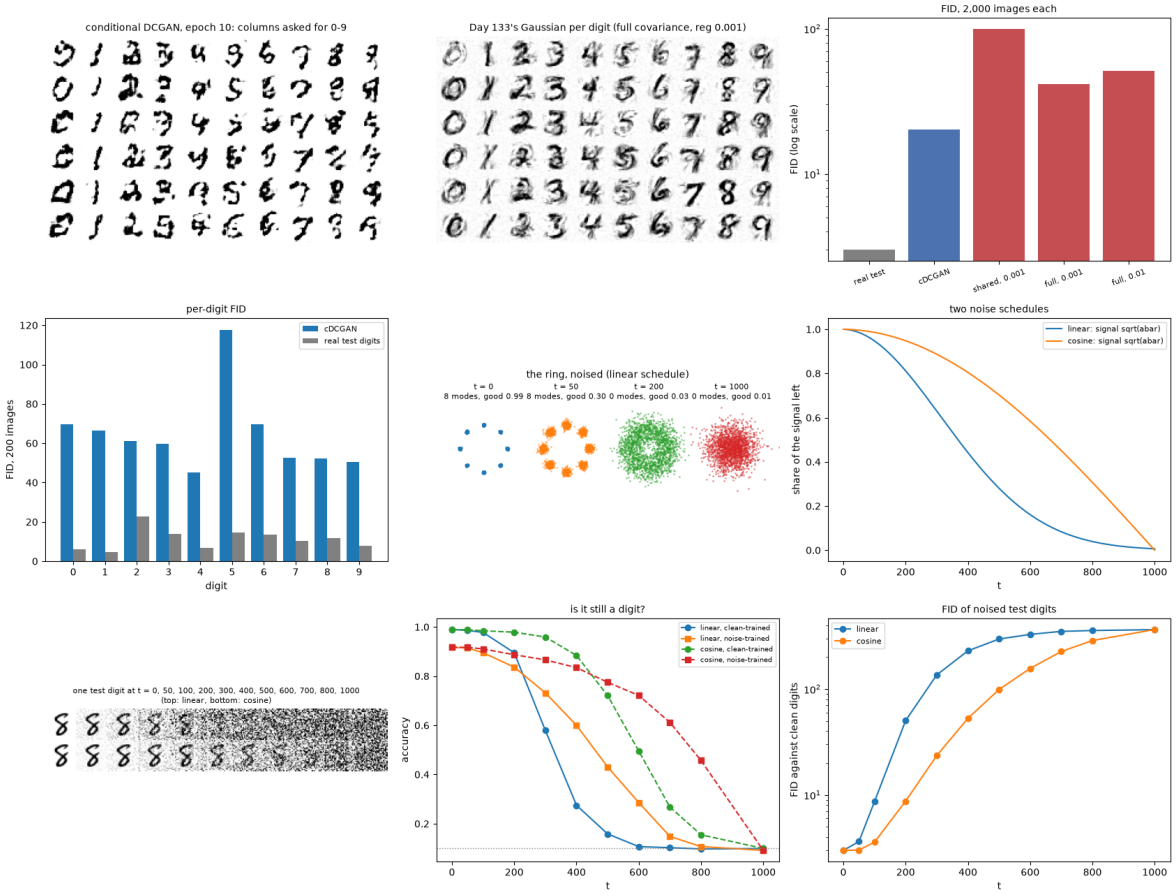
demo_mnist_noise: where accuracy halves

CODE

```
half = next((t for t, _, _, na, _, _ in rows if na < 0.5), None); print(f"   the noise-trained
    accuracy first falls below 0.5 at t = {half}")
```

- `next(generator, None)` — the first item the generator yields, or None if it yields nothing.
- `for t, _, _, na, _, _ in rows` — unpacks each six-item row, keeping only t and the noise-trained accuracy.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `sys.modules[__name__]` — the module object of the running file.
- `ast.parse(src).body` — a file's top-level statements, as syntax-tree nodes.
- `np.cumprod(a)` — running product.
- `a[1:] / a[:-1]` — each element divided by the one before it.
- `next(gen, default)` — the first item of a generator, or a default.
- `f(*pair, ...)` — unpack a tuple into separate arguments.

End of Day 139 syntax reference. For the experiments and what they showed, see the Day 139 complete guide.